

$w_{ij}^{(k)}$: weight is coming from i , going to j and it is in layer no (k)

alternate
node
edge

→ For weight update rule can be $w = w - \eta \frac{\partial C}{\partial w}$

Change in cost with respect to weights.

→ In case of bias $b_i = b_i + \Delta b_i$

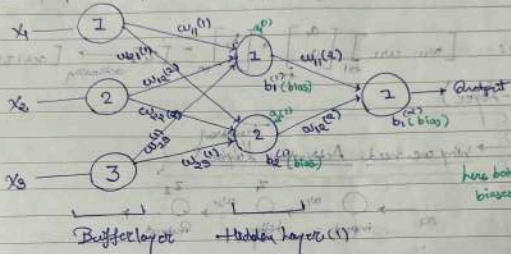
where $\Delta b_i = -\eta \frac{\partial C}{\partial b_i}$

called as bias rule also.

→ * ANN:

Input layer

Here Input layer is Output layer is total weights 8×6



here both the biases are diff.

→ When we says, model is training, it means we update weights and bias.

In hidden layer (1):

$$\therefore w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + w_{13}^{(1)} x_3 + b_1^{(1)} = z_1^{(1)}$$

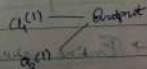
→ Here we have multiple variations. Note these variations, that's important too.

Output as a_1 :

$$\therefore a_1^{(1)} = \sigma(z_1^{(1)})$$

$$\text{Also, } w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + w_{23}^{(1)} x_3 + b_2^{(1)} = z_2^{(1)}$$

$$\text{Output as } a_2: a_2^{(1)} = \sigma(z_2^{(1)})$$



Now, In final Output layer:

$$z_1^{(2)} = w_{11}^{(2)} \cdot a_1^{(1)} + w_{12}^{(2)} \cdot a_2^{(1)} + b_1^{(2)}$$

$$\text{Final Output: } a_1^{(2)} = \sigma(z_1^{(2)}) = y$$

Linear Combination in a form of impulse response

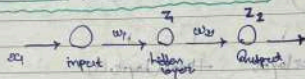
→ Before and between input and output layer

Layer 1:
$$\begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \xrightarrow{\text{activation}} \begin{bmatrix} \text{act}(z_1) \\ \text{act}(z_2) \end{bmatrix}$$

Layer 2:
$$\begin{bmatrix} w_{11} & w_{12} \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} + \begin{bmatrix} b_1 \end{bmatrix} = \begin{bmatrix} z_1 \end{bmatrix} \xrightarrow{\text{activation}} \begin{bmatrix} \text{act}(z_1) \end{bmatrix} \rightarrow \hat{y}$$

(Output-Layer)

→ why we need Activation Layer?



Let's assume we are not using activation layer function at all.

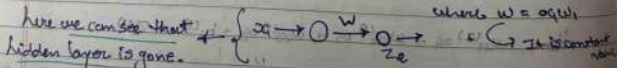
Output will be, $z_1 = x_1 w_1$ and $z_2 = z_1 w_2$

$$\therefore z_2 = z_1 \cdot w_2$$

$$= (x_1 w_1) \cdot w_2$$

$$\therefore z_2 = w_1 \cdot w_2 \cdot x_1$$

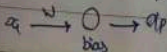
here we can see that hidden layer is gone.



→ Entire network will convert just a single neural network. That's why,

we are using Activation Function.

With bias:



$$z = w \cdot x + \text{bias}$$

Some constant
constant

Output : activation (dot(input, weight) + bias)

For example

If there is no Activation Function, all the weights and bias are just going to blow down through a model unrestricted, as whole model is the single neuron.

$$\begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} & w_{13}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} & w_{23}^{(1)} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} z_1^{(1)} \\ z_2^{(1)} \end{bmatrix}$$

$$\begin{bmatrix} w_{11}^{(2)} & w_{12}^{(2)} \end{bmatrix} \begin{bmatrix} z_1^{(1)} \\ z_2^{(1)} \end{bmatrix} = z_1^{(2)} + w_{11}^{(2)} z_1^{(1)} + w_{12}^{(2)} z_2^{(1)}$$

$$\rightarrow z_1^{(2)} = w_{11}^{(2)} z_1^{(1)} + w_{12}^{(2)} z_2^{(1)}$$

$$= w_{11}^{(2)} \cdot \{ w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + w_{13}^{(1)} x_3 \} + w_{12}^{(2)} \cdot \{ w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + w_{23}^{(1)} x_3 \}$$

$$= \{ A x_1 + B x_2 + C x_3 \} + \{ P x_1 + Q x_2 + R x_3 \}$$

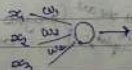
where $A = w_{11}^{(2)} \cdot w_{11}^{(1)}$ same constant

$B = \dots$

$$= (A+P) x_1 + (B+Q) x_2 + (C+R) x_3$$

$$z_1^{(2)} = w_1 x_1 + w_2 x_2 + w_3 x_3$$

it's look like



Complex Neural Network also has converted into single Neuron.

Diff Activation layer : Sigma, sigmoid, ReLU, Softmax

Softmax:



For example

$$e^{a_1} + \dots + e^{a_n} = \frac{1}{2} = 2$$

$$\text{where } p_i = \frac{e^{a_i}}{e^{a_1} + e^{a_2} + e^{a_3}}$$

will give probability.

$$\text{Softmax function: } \frac{e^i}{e^1 + e^2 + e^3}$$

For example - You can find many on all use Softmax function as activation layer

- Train : $\rightarrow$ some freq.
- Valid : $\rightarrow$ train and test
- Test : $\rightarrow$ train and valid

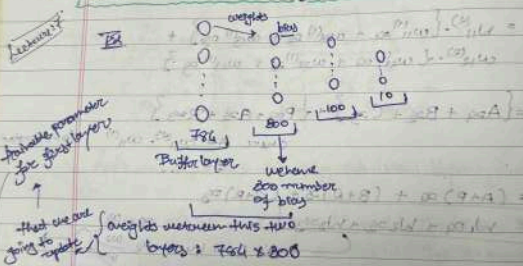
$\rightarrow$ We are talking about weights and bias which is getting updated

$\frac{W}{\text{Matrix}} \quad \frac{b}{\text{Matrix}}$

$\left. \begin{array}{l} \text{each data point of weight is occupying} \\ \text{more memory} \end{array} \right\}$

Suppose
we have increased weights and number of biases, so it will occupy more memory
 $\rightarrow$ so training process will be slow.

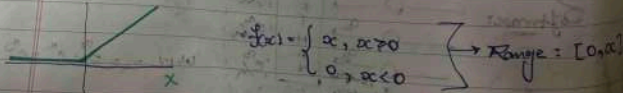
How to calculate number of trainable parameters?



$\rightarrow$ also we have 800 numbers of output biases:

$= 784 \times 800 + 800$

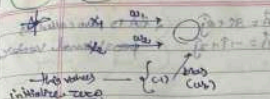
Activation Function : Relu:



ReLU: Rectified Linear Activation Unit

Rectifying (this behavior like a line only)

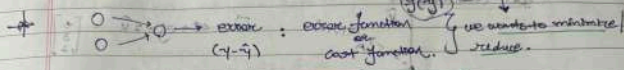
In softmax function, as output whichever output is getting more probability, then we can say that it is predicting as that class.



Cost Function:

- $\rightarrow (y - \hat{y})^2$
- $\rightarrow (y - \phi(w \cdot x + b))^2$
- $\rightarrow \text{Cost Function} = C(w, b)$

If I will get error then all weights w , bias and b will be changed.



Loss Function: Sparse Categorical Crossentropy

$J(x)$
 Come output directly depends on some weights and bias.

SGD: Stochastic Gradient Descent

weight update formula:

$w = w - \eta \frac{\partial C}{\partial w}$ - derivative of cost function with respect to weights.

$b = b - \eta \frac{\partial C}{\partial b}$

Vector:

Magnitude + Direction

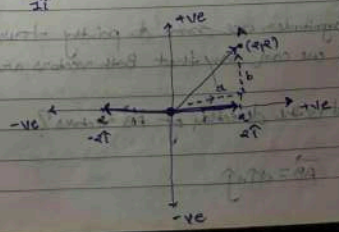
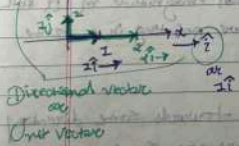
eg.

I am moving 5 km/h in various directions. Speed is 5 km/h



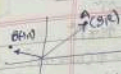
Why unit vectors?

Their magnitude is considered as 1 and they have specific direction.



like, first to reach point A like we have to go with a then b then we can reach point A.

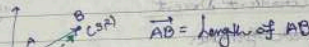
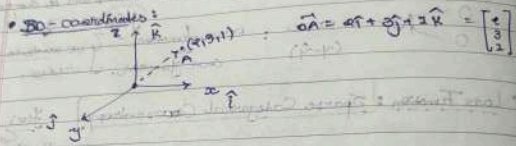
$\vec{OA} = a\hat{i} + b\hat{j}$



they, we can say that $\vec{OA} = 2\hat{i} + \hat{j}$
 $\vec{OB} = -1\hat{i} + 2\hat{j}$

Also, $\vec{OA} = 2\hat{i} + \hat{j}$
 $= \begin{bmatrix} 2 \\ 1 \end{bmatrix}$

$\vec{OA}$ is also called as positional vector

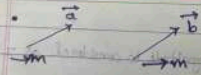


$\vec{OA} = 2\hat{i} + \hat{j}$
 $\vec{OB} = \hat{i} + 2\hat{j}$

We can say that $\vec{AB} = \vec{OB} - \vec{OA}$ (Final - Starting point)

$= (\hat{i} + 2\hat{j}) - (2\hat{i} + \hat{j})$
 $\vec{AB} = -\hat{i} + \hat{j}$

Length / Magnitude of vector: $|\vec{AB}| = \sqrt{(-1)^2 + (1)^2} = \sqrt{2}$



Both vectors has magnitude m & both of them are pointing in same direction

Equal Vectors

→ Magnitudes are same & pointing towards same direction
 then we can say that Both vectors are same.

Now What is the direction of $\vec{AB}$ vectors?

$\vec{AB} = -\hat{i} + \hat{j}$

Vector Direction : magnitude $\times$ Unit Vector

Unit vector: vector which gives a direction.
(magnitude = 1)

→ We can write as $\vec{AB} = \hat{a}$

→ a unit vector in direction of $\vec{AB}$.

$$\vec{AB} = \hat{a} = \frac{\vec{AB}}{|\vec{AB}|}$$

$$= \frac{a\hat{i} + b\hat{j}}{\sqrt{a^2 + b^2}} = \frac{a\hat{i} + b\hat{j}}{\sqrt{5}}$$

way of finding
a unit vector
in direction of
 $\vec{AB}$

Suppose magnitude = 5 then:

Vector pointing in
direction of $\hat{a} = 5 \cdot \hat{a}$

$$= 5 \cdot \left(\frac{a\hat{i} + b\hat{j}}{\sqrt{5}} \right)$$

→ Unit vectors are just directional vectors, which are used for giving a

Suppose:

$\vec{r} = \vec{AB}$ is given and I want unit vector in direction of $\vec{AB}$

then simply I will divide it by $|\vec{r}|$.

$$\hat{a} = \frac{\vec{r}}{|\vec{r}|} = \frac{\vec{r}}{r}$$

i.e. I am dividing entire vector with its only magnitude.

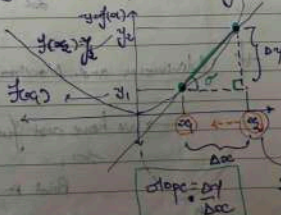
i.e. we are removing magnitude.

Differentiation:

Conceptually → slope of tangent at a point on a graph

Small change in $y = \frac{dy}{dx}$

Small change in x



Slope also represents as,

Slope = $\frac{\text{rise}}{\text{run}}$

= $\frac{\text{perpendicular}}{\text{base}}$

$$\text{Slope} = \frac{\Delta y}{\Delta x}$$

$$\text{Slope} = \frac{y_2 - y_1}{x_2 - x_1}$$

Imp
We say slope because we are finding a point where curve starts to rise. To find that point where slope starts to rise.

→ If I want to reduce Δx then I keep writing as $\lim_{\Delta x \rightarrow 0}$.

ie if Δx becomes zero

ie we can say that as point reaching towards or direction.

So, we can say that with Δx , dy value will also be changed.

So, we can say,

$$\lim_{\Delta x \rightarrow 0} \frac{\Delta y}{\Delta x} = \frac{dy}{dx}$$

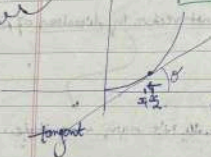


derivative
is equal to
comp. value

$$\Rightarrow \lim_{\Delta x \rightarrow 0} \frac{y_2 - y_1}{\Delta x} = \lim_{\Delta x \rightarrow 0} \frac{f(x_2) - f(x_1)}{\Delta x}$$

$$\frac{dy}{dx} = \lim_{\Delta x \rightarrow 0} \frac{f(x_1 + \Delta x) - f(x_1)}{\Delta x}$$

from diagram if you see



If x_1 and x_2 are very close, then we can say that graph will look like...

$m = +ve$ or $-ve$

For $\tan \theta$, if $0^\circ < 90^\circ$ then

$$\tan \theta = +ve \quad (0, 90)$$

$\tan \theta$, if $90^\circ < \theta$ then

$$-\tan \theta = -ve \quad (90, 180)$$

$$y = x^2$$

$$y = x^n$$

$$\frac{dy}{dx} = 2x$$

$$\frac{dy}{dx} = n \cdot x^{n-1}$$

Minimum and Maximum of a function:

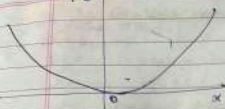
→ We have cost function / error function, whose graph is like



Point A is minimum value and slope of that point

Derivative: Minimum of any error.

$y=f(x)=x^2$



Step 1: Find differentiation of function: $\frac{dy}{dx} = 2x$

Step 2: $\frac{dy}{dx} = 0$ to find x value:

$\frac{dy}{dx} = 2x = 0 \Rightarrow x = 0$

Step 3: Again find diff of y , i.e. $\frac{d^2y}{dx^2}$

i.e. $\frac{d}{dx}(2x) = 2 > 0$

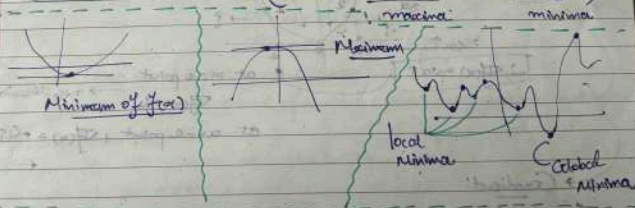
Called as Successive Differentiation

If $\frac{d^2y}{dx^2} > 0$ at $x=0$, then point of Minimum

If $\frac{d^2y}{dx^2} < 0$ at $x=0$, then point of Maximum

i.e. $x=0$ is Minimum of $f(x)$. I can verify from graph.

If $\frac{d^2y}{dx^2} = 0$ then it is called as Saddle point / point of inflection or (neither maximum nor minimum)



Optimum: Max or Min i.e. $y=f(x)$

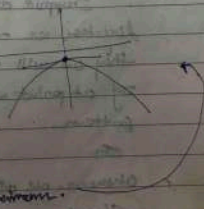
Optimize

$\frac{dy}{dx} = -2x$

$\frac{d^2y}{dx^2} = -2 = 0$

$\therefore -2 < 0$

Maximum



Hence proved.

Partial Derivatives:

$$\rightarrow f(x, y) = x^2 + y^2$$

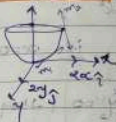
$$\frac{\partial f}{\partial x} = 2x = 0, \quad \frac{\partial f}{\partial y} = 2y$$

Gradient (∇):

$$\rightarrow \nabla f(x, y) = \frac{\partial f}{\partial x} \hat{i} + \frac{\partial f}{\partial y} \hat{j} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

$$\text{let } f(x, y) = x^2 + y^2$$

$$\rightarrow \nabla f(x, y) = 2x \hat{i} + 2y \hat{j}$$

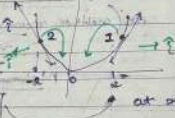


$$\text{let } f(x) = x^2$$

$$\rightarrow \nabla f(x) = \frac{\partial f}{\partial x} \hat{i}$$

$$= 2x \hat{i}$$

$$|\nabla f(x)| = 2|x|$$



at $x = e$ point

$$\nabla f(x) = 2\hat{i}$$

at $x = -e$ point

$$\nabla f(x) = -2\hat{i}$$

← -ve direction

Gradient:

Suppose at point 1, where would you like to move, towards the direction or -ve direction? - Towards Negative. at point 2 they will reach ✓ this direction somewhere at the bottom. If at point 2, will reach this ✓ direction somewhere at the bottom.

So,

Whenever we will calculate ∇f , it will always gives us a direction towards the steepest ascents,

Meaning, gives a direction where pointing towards increasing of f .

i.e. Gradient ascent.

Optimizing

Point 1 is pointing towards $\rightarrow$ +ve direction, Suppose I am moving that direction, $\alpha = 0.01$, it will keep on increasing slope value.

i.e. Gradient Ascent.

But I want to move opposite direction: So I will keep -ve sign $(-\alpha f)$.

Point 2, it is pointing towards $\leftarrow$ -ve direction, It will keep on increasing slope, it won't reach at the bottom, where is my goal (0). So Solution is put -ve sign $(-\alpha f)$.

Gradient Descent: We want descent towards minimum.

i.e. it is opposite of Ascent.

In short, cost function: $(y - \hat{y})^2 = f$

$$\Rightarrow (y - \phi(\vec{x}))^2$$

$$\Rightarrow (y - \phi(w_0x + b))^2$$

$$\Rightarrow \text{cost function} : C(w, b)$$

depends on weights and bias.

Objective: Minimize / optimize cost function.

For that we need some optimizer - which will give us next new

or updated weights.

$$\text{i.e. } W = W + \Delta W \text{ and } b = b + \Delta b$$

$$\therefore \Delta W = -\eta \frac{\partial C}{\partial W}$$

$$= -\eta \frac{\partial C}{\partial W}$$

cost function

Gradient Descent

$$\therefore \Delta W = -\eta \frac{\partial C}{\partial W}$$

This is why in formula we have -ve sign.

Again, Suppose, cost function $f = (y - \hat{y})^2$

$$= (y - (\phi(w_0x + b)))^2$$

Let us

assume that $y = k$ as it is an actual value. also it is constant

Also assume that, bias = 0 & 0
 $\therefore \phi(w \cdot x + b) \rightarrow \phi(w \cdot x)$

If $\phi = \text{ReLU function}$ then

$$\phi(w \cdot x) = \begin{cases} w \cdot x, & w \cdot x \geq 0 \\ 0, & w \cdot x < 0 \end{cases}$$

If $\phi = \text{Sigmoidal function}$ then

$$\sigma(w \cdot x) = \phi$$

i.e. Domain for ϕ will be $(-\infty, \infty)$ and range is $(0, 1)$

To,

Here we can observe that value of ϕ is depending on activation function. So we can say

ϕ will return some value which depends on weights only. That what we are updating, we are looking for change.

Let $\phi(w \cdot x)$ will be some kind of fixed value w .

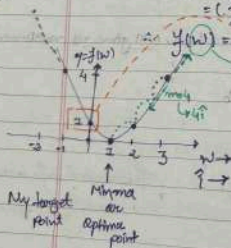
$$J = (y - \phi(w \cdot x))^2$$

$$= (y - w)^2$$

$$(\because w = \phi(w \cdot x))$$

$$J(w) = (y - w)^2$$

now, it becomes the function of w , what we are looking for



(1) Randomly initialize a weight.

$$\text{Case: } 1, w = 3$$

$$J(3) = 4.$$

Now, we need to calculate gradient of cost function

$$\nabla J(w) = \frac{\partial J}{\partial w}$$

$$\text{Now, } \nabla J(w=3) = 2w - 2$$

$$\therefore \nabla J(w=3) = 6 - 2 = 4$$

$$= 4$$

$$\frac{\partial J}{\partial w} = 2(1 - w)(-2)$$

$$= -2(2w)$$

$$\therefore \frac{\partial J}{\partial w} = 2w - 2$$

$$\nabla f(w=3) = 4$$

Represents, $\left(\frac{\partial f}{\partial w}\right)_{w=3}$ which is a slope of the tangent to the function f at $w=3$ point.

is at that point $w=3$.

i.e. $4 \uparrow$, it is pointing away from the point.

Means, if I will move $\rightarrow$ this direction so in curve I will move up side and I won't get minimum point.

If I will go -ve direction then I can reach at the bottom.

So,

By descent gradient gives up ascent value which means up side of curve. i.e. moving away from the minimum point.

(ii) Update the weights.

$\rightarrow$ we have $\nabla f(3) = 4$

$$If \ w = w + \Delta f(w)$$

$$= 3 + (4)$$

$$w = 7$$

So, I should add negative to this.

$$i.e. \ w = w - \nabla f(3)$$

$$= 3 - (4)$$

$$w = -1 \quad \text{But I wanted to reach at 1}$$

So I will go up side I don't want to reach at minimum point.

Gradient Ascent

Here I am overshooting my target. If I will multiply

Something with $\nabla f(w)$ then we can adjust it $\rightarrow$ Learning rate.

$$w = w - \eta \nabla f(w)$$

Case: 2

Let $w = -2$

$$f(-2) = (1 - (-2))^2 = 9$$

$$\text{also, } \left(\frac{\partial f}{\partial w}\right)_{w=-2} = 2w - 2$$

$$= 2(-2) - 2 = -6$$

$$w = w - \eta \nabla f(w)$$

$$= -2 - (0.1)(-6) = -1.4 \quad \text{So, it is towards bottom point i.e. we are in correct position.}$$

away from bottom.

Chain Rule:

Let $h(x) = \frac{1}{1+e^{-x}}$, $\frac{dh}{dx} = ?$
 Let $p = -x$ and $q = \frac{1}{1+e^p} = \frac{1}{1+e^{-x}}$
 $\therefore \frac{dq}{dp} = -\frac{e^p}{(1+e^p)^2}$

$\frac{dy}{dx} = \frac{dy}{dq} \cdot \frac{dq}{dp} \cdot \frac{dp}{dx}$

$\frac{dy}{dq} = -\frac{q}{1-q}$, $\frac{dq}{dp} = -\frac{e^p}{(1+e^p)^2}$, $\frac{dp}{dx} = -1$

$\therefore \frac{dy}{dx} = \frac{dy}{dq} \cdot \frac{dq}{dp} \cdot \frac{dp}{dx}$

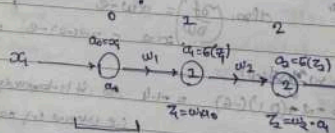
$= -\frac{1}{q^2} \cdot e^p \cdot (-1) = \frac{e^p}{1+e^p} = \frac{e^{-x}}{(1+e^{-x})^2}$

$= \frac{(1+e^x)-1}{(1+e^x)^2} = \frac{1}{(1+e^x)^2} = \frac{1}{(1+e^{-x})^2}$

$\frac{1}{(1+e^{-x})^2} \left[\frac{1}{1+e^{-x}} - \frac{1}{1+e^{-x}} \right] = f(x) [1-f(x)]$

Back-Propagation:

Derivative of Sigmoid function



Input layer

$y = \text{actual value}$
 Error = $(y - \hat{y})^2$

Assumption: Each layer has 1 neuron

or As of now bias is zero.

As we know, weights are updated based on $\frac{\partial e}{\partial w}$

$$e = w - y \quad \text{cost function}$$

$$= w - y \frac{\partial e}{\partial w}$$

So, firstly our objective is to find out $\frac{\partial e}{\partial w}$

As we know, $e = (y - \hat{y})^2$

$$e = (y - a_1)^2$$

can say that e is depending on a_1

Also, we know

$$a_1 = \sigma(z_1), \text{ where } z_1 = w_1 a_0$$

Here a_1 is dependent on z_1 and z_1 is dependent on a_0

Objective: How much this error (e) is contributing back of w_1 .

$$\text{i.e. } \frac{\partial e}{\partial w_1} = \frac{\partial e}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

$$\frac{\partial e}{\partial a_1} = -2(y - a_1)$$

$$\text{Now, } w_1 = w_1 - \eta \cdot \frac{\partial e}{\partial w_1}$$

$$\frac{\partial a_1}{\partial z_1} = \sigma(z_1) [1 - \sigma(z_1)]$$

This entire process was about weight w_1 $\rightarrow \frac{\partial z_1}{\partial w_1} = a_0$

$$e = (y - a_1)^2 \rightarrow a_1 = \sigma(z_1) \rightarrow z_1 = w_1 a_0 \rightarrow$$

Now, we want to concentrate on weight a_1 then,

$$e = (y - a_1)^2 \rightarrow a_1 = \sigma(z_1) \rightarrow z_1 = w_1 a_0 \rightarrow a_1 = \sigma(z_1) \rightarrow z_1 = w_1 a_0$$

Here:

a_1 is dependent on z_1 , z_1 is on a_0 , a_0 is on z_0 , z_0 is on a_{-1}

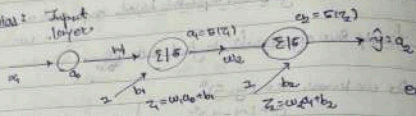
$$\frac{\partial e}{\partial w_1} = \frac{\partial e}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial a_0} \cdot \frac{\partial a_0}{\partial z_0} \cdot \frac{\partial z_0}{\partial a_{-1}}$$

$$= (-2(y - a_1)) \cdot (\sigma(z_1) - (1 - \sigma(z_1))) \cdot w_1 \cdot (\sigma(z_0) - (1 - \sigma(z_0)))$$

Change in error with respect to w_1

weight update: $w_1 = w_1 - \eta \frac{\partial \text{cost}}{\partial w_1}$

Node, With bias: Input layer



error: $e = (y - \hat{y})^2$
 $C = (y - a_2)^2 \rightarrow f(a_2)$

Node, $a_2 = G(z_2) \rightarrow f(z_2)$
 $z_2 = a_1 w_2 + b_2 \rightarrow f(w_2, a_1, b_2)$

Weights: $w = w - \eta \frac{\partial e}{\partial w}$ and bias: $b = b - \eta \frac{\partial e}{\partial b}$

$\frac{\partial e}{\partial w_1} = \frac{\partial e}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_1} \cdot \frac{\partial z_1}{\partial w_1}$
 $= () \cdot a_1$

$\frac{\partial e}{\partial b_1} = \frac{\partial e}{\partial a_1} \cdot \frac{\partial a_1}{\partial b_1} \cdot \frac{\partial z_1}{\partial b_1}$
 $= () \cdot 1$

$\therefore w_1 = w_1 - \eta \frac{\partial e}{\partial w_1}$

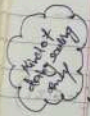
$\therefore b_1 = b_1 - \eta \frac{\partial e}{\partial b_1}$

Similarly, we can find $\frac{\partial e}{\partial w_2}$ and $\frac{\partial e}{\partial b_2}$.

$w_1: e \rightarrow a_1 \rightarrow z_1 \rightarrow a_2 \rightarrow z_2 \rightarrow w_2$ $b_1: e \rightarrow a_1 \rightarrow z_1 \rightarrow a_2 \rightarrow z_2 \rightarrow w_2$

(Python Implementation)

Activation Function:



Activation Function helps to determine the output of a neural network. These types of functions are attached to each neuron in network, and determines whether it should be activated or not, based on whether each neuron's input is relevant for model's prediction.

→ helps to normalise output of each neuron to range between 1 and 0 or -1 and 1.

* Problems faced in training Neural Network

(1) Vanishing and Exploding gradient. (Common issue)

Vanishing: when gradient becomes very small.

Exploding: when gradient becomes very huge.

→ Training of layers results: unstable gradients.
i.e. problem will not solve.

(2) It requires lot of data to train.

→ can be solved by transfer learning or data augmentation technique.

(3) Whenever we will increase the size of neural network, that means increasing number of hidden layers → slow training.

→ Solution: we can go for better optimizers or can choose better activation function.

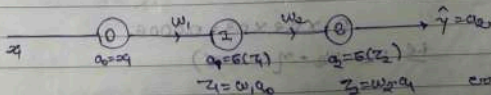
(4) Risk of overfitting. (Common)

→ not enough data.
→ noisy data.

→ Millions of parameters, → there will be lots of trainable parameters.
if we have huge no of parameters & less number of dataset for source it will learn very easily.

Solution: Some regularization techniques, Dropout layer.

* Vanishing and Exploding Gradient:



Vanishing Gradient
or weight initialization. (Both are randomly initialized it)
depends on

Change in error with respect to weight w_2 is given by

$$\frac{dC}{dw_2} = \frac{dC}{da_2} \cdot \frac{da_2}{dz_2} \cdot \frac{dz_2}{dw_2} \quad (1)$$

Change in error with respect to weight w_1 is given by

$$\frac{dC}{dw_1} = \frac{dC}{da_2} \cdot \frac{da_2}{dz_2} \cdot \frac{dz_2}{da_1} \cdot \frac{da_1}{dz_1} \cdot \frac{dz_1}{dw_1} \quad (2)$$

Here, we have look on derivative of activation function during the process of their respective weights.

→ We are dependent on activation function, so rate of activation function is important over there.

Box:

We choose diff activation function, value of a_2 will change.

So

We can observe that, eq (1) & (2) both the derivatives are dependent on activation function, as well as, error is also dependent on choice of the output initialization technique.

Also, can say both the equations are kind of ratios only.

→ If in both equations, all individual ratios are between 0 and 1 then weight update will be almost very small.

eg

Let in eq (1), $dc/da_2 = 0.1$, $da_2/dz_2 = 0.2$, $dz_2/dw_2 = 0.3$

then,

$$0.1 \times 0.2 \times 0.3 = 0.006$$

$$\text{i.e. } w_2 = w_2 - \eta (0.006)$$

i.e. almost no change or very less change in w_2 .

That is called as Vanishing Gradient.

Understanding the difficulty of training deep feed forward neural networks

One major difficulty is vanishing gradient

While Vanishing Gradient: (losses can be fast)

→ Lower layers are hard to train

→ Training will be very slow

→ If in both the equations, weights of these product terms are > 1 , then weight updates will be very high.

Suppose, in eq (1)

$$\text{Let } w_1/w_2 = 10, w_3/w_4 = 20 \text{ and } w_5/w_6 = 30$$

$$\text{then } 10 \times 20 \times 30 = 6000$$

$$\text{i.e. } w_6 = w_1 \times 6000$$

i.e. still values can be almost high

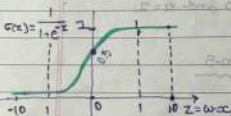
This is called as Exploding Gradient.

we can say that it is kind of unstable gradients. It will update the weights in a very huge manner.

In Exploding Gradient: Solution will diverge, means we won't ever reach our actual solution.

Also, we can observe here that, each layer learns at different speed.

Observation with respect to Sigmoid Function:



Assume that bias = 0.

Case: I, Initialize weight randomly, i.e. $w = 10$

Also, $x = 1$.

$$\therefore z = w \cdot x = (10)(1) = 10$$

$$\therefore G(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-10}} = \frac{1}{1 + \frac{1}{e^{10}}} = \frac{1}{1 + \frac{1}{31622.7766}} \approx \frac{1}{1.0000316} \approx 1$$

$$\therefore \frac{\partial G(z)}{\partial z} = G(z)[1 - G(z)]$$

($\because$ activation function)

$$\therefore \frac{\partial G(10)}{\partial z} = 1(1 - 1) = 0$$

It is between 0.1
So mean value
will be 0.5

From above eq (17): $\frac{dw}{dz} = \frac{dw}{dz} \cdot \frac{dz}{dz}$ (Note: $\frac{dz}{dz} = 1$ because z is a scalar)

$$\therefore \frac{dw}{dz} = 0$$

i.e. Gradient is zero.

$$\text{i.e. } w = w - \eta \frac{dw}{dz}$$

$$\therefore w = w - 0 \Rightarrow w = w \Rightarrow \text{No change in weight.}$$

So, we can observe that

If I choose very huge numbers, i.e. very positive numbers, in z , then weight updates will be very less.

Case 2: Initialize weight randomly, $w = -10$ and $a = 1$.

$$\therefore z = w \cdot a = -10$$

$$\therefore G(z) = G(-10) = \frac{1}{1 + e^{-(-10)}} = \frac{1}{1 + e^{10}} \approx 0.$$

$$\text{i.e. } \frac{\partial G(z)}{\partial w} = G(z)[1 - G(z)]$$

$$= (00)[1 - 0] = 0.$$

Again, weight updates will be zero.

Case 3: Initialize weight randomly, $w = 0$ and $a = 1$.

$$\therefore z = w \cdot a = 0$$

$$\therefore G(z=0) = \frac{1}{1 + e^0} = \frac{1}{2} = 0.5$$

$$\therefore \frac{\partial G(z=0)}{\partial w} = 0.5[1 - 0.5] = 0.25$$

$$\therefore w = w - \eta \frac{\partial G}{\partial w}$$

(here we can see that there will be some change in weight)

This is why weight initialization is also important with choice of activation function.

6

part of the world distribution

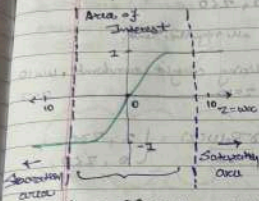
classmate

Page No.

Hyperbolic Tangent

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

It is between -1 and +1. Mean value is 0.



Now

$$\begin{aligned} \frac{\partial \tanh(z)}{\partial z} &= \frac{(e^z + e^{-z})(e^z - e^{-z}) - (e^z - e^{-z})(e^z + e^{-z})}{(e^z + e^{-z})^2} \\ &= \frac{(e^{2z} + 1) - (e^{2z} - 1)}{(e^z + e^{-z})^2} \\ &= \frac{2}{(e^z + e^{-z})^2} \end{aligned}$$

$$\frac{\partial \tanh(z)}{\partial z} = 1 - \tanh^2(z)$$

Case:1 Initialize weight randomly, $w=10$ and $x=1$

then $z = w \cdot x = 10$

$$\therefore \tanh(10) = 1 \quad (\text{From the graph})$$

$$\therefore \frac{\partial \tanh(z)}{\partial z} = 1 - (\tanh(z))^2$$

$$= 1 - (1)^2$$

$$= 0$$

Case:2 Initialize weight randomly, $w=-10$ and $x=1$

then $z = w \cdot x = -10$

$$\therefore \tanh(-10) = -1 \quad (\text{From the graph})$$

$$\therefore \frac{\partial \tanh(z)}{\partial z} = 1 - (-1)^2$$

$$= 1 - 1 = 0$$

Case:3 Initialize weight randomly, $w=0$ and $x=1$

then $z = w \cdot x = 0$

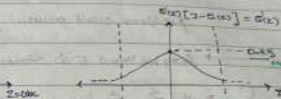
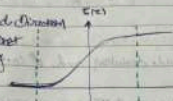
$$\therefore \tanh(0) = 0 \quad (\text{From the graph})$$

$$\therefore \frac{\partial \tanh(z)}{\partial z} = 1 - (0)^2$$

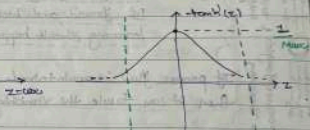
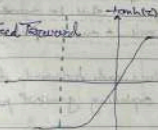
$$= 1$$

Observation: Table: Derivatives in paper 1

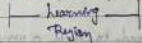
• Forward Direction
 otherwise predict something.



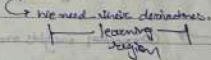
• Feed Forward



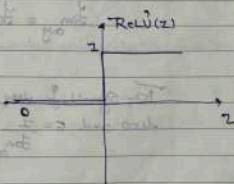
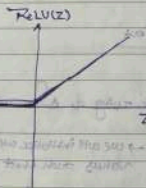
Forward Direction Case $\rightarrow$



Backward Direction



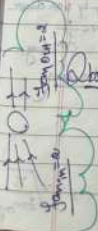
• ReLU:



Observation:

Incoming
 f_{in} : Number of edges coming to the input layer.

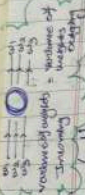
Outgoing
 f_{out} : Number of outgoing edges from a layer.



→ If $\dot{z}_{in} = \dot{z}_{out}$ then vanishing and Exploding gradient problem won't occur.

→ Also, we know σ as Standard deviation and σ^2 is Variance.

→ If $\dot{z}_{in} = \dot{z}_{out}$ then vanishing and Exploding gradient problem won't occur.



Objective: The flow of information / Input data should easy pass i.e. forward and backward process should be smooth. i.e. Learning should happen in each & every epoch.

If proper flow is maintained in forward and backward direction then it can tackle the vanishing and exploding gradient problems.

→ This things can not be achieved every time, it is not possible. Since we know if we proceed to result, final layer, one of weights will decrease.

→ Connection weights must be initialized in such a way, that

$$\dot{z}_{avg} = \frac{\dot{z}_{in} + \dot{z}_{out}}{2}$$

For Sigmoid function, if we are using a Normal distribution with $\mu=0$ and $\sigma = \frac{1}{\dot{z}_{avg}}$

→ we will initialize weights, randomly with values such that $\mu=0$ and $\sigma = \frac{1}{\dot{z}_{avg}}$

- Called as Glorot Initialization / Xavier Initialization

→ It is the By default case in Keras's Dense layer.

→ Uniform Distribution: between $-\sigma$ and $\sigma \rightarrow \sigma = \frac{\sqrt{3}}{\dot{z}_{avg}}$

6

General Normal Distribution: $f(x) = \frac{1}{\sigma\sqrt{2\pi}} \cdot e^{-\frac{(x-\mu)^2}{2\sigma^2}}$



Standard Normal Distribution: $f(x) = \frac{1}{\sqrt{2\pi}} \cdot e^{-\frac{x^2}{2}}$



By	Initialization	Activation Function	σ^2 (normal)
Glenn	Glenn	None, tanh, Sigmoid, etc.	$\frac{1}{\text{fan}}$
He	He	ReLU & it's variants,	$\frac{2}{\text{fan}}$
LeCun	LeCun	SELU	$\frac{1}{\text{fan}}$

While coding

In keras: One parameter is "kernel_initializer = ", then we have to provide these values like, glorot_uniform.

Sigmoid Function:

First draw graph: $\sigma(x)$ and $\sigma'(x)$

- The sigmoid function is the most frequently used activation function in beginning of deep learning.
- It is smoothing function that is easy to derive.
- In sigmoid function output is in open interval (0,1)

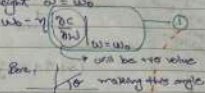
Function itself has certain defects:

- When the input is slightly away from the coordinate origin, the gradient of the function becomes very small, almost zero.
- In the process of neural network backpropagation, we all use the chain rule of differential to calculate of each weight w.



So, initial weight $w_0 = w_0$

$$w_1 = w_0 - \eta \left(\frac{\partial J}{\partial w} \right)_{w=w_0}$$



For making this angle

Also η will be +ve so, $\eta \left(\frac{\partial J}{\partial w} \right)$ will be overall +ve.

Also, w_0 is positive value.

$$w_1 = w_0 - (\text{tile})$$

$$\text{Now, } w_2 = w_1 - \eta \left(\frac{\partial J}{\partial w} \right)_{w=w_1}$$

Again same process, everything will be the same which will be high value $\eta > 2$

So we can observe that, at every jump we will get steeper (smaller value)



Observation: $\eta = \text{constant}$

change in weights Δw & $\frac{\partial J}{\partial w}$

i.e. if Δw is high then $\frac{\partial J}{\partial w}$ will be more steep.

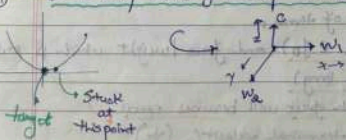
if Δw is low then $\frac{\partial J}{\partial w}$ will be flat or closer to local minima or maxima.

② Problem: It will stuck at saddle point.

It will stuck at areas where slope is very less

③ It doesn't depend upon or consider past weight updates.

④ It is very slow for deep neural networks.



Suppose we have two weights.

In that case how this gradient will be change?

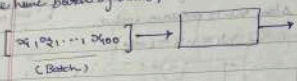
new bias only one (bias) term will be added.

Ans:
$$\begin{bmatrix} w_1 \\ w_2 \end{bmatrix}_{\text{new}} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}_{\text{old}} - \eta \left[\frac{\partial C}{\partial w_1} \quad \frac{\partial C}{\partial w_2} \right]$$
 $w = \text{old weights}$

$$\vec{w}_{\text{new}} = \vec{w}_{\text{old}} - \eta \nabla C(w_1, w_2)$$

$$(w_1 + w_2)_{\text{new}} = (w_1 + w_2)_{\text{old}} - \eta \left(\frac{\partial C}{\partial w_1} + \frac{\partial C}{\partial w_2} \right)$$

Let us have batch of data,



So, it will calculate MSE

$$\frac{1}{2} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

objective is to reduce

overall cost function.

Cost function

$$J = \frac{1}{2} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Momentum Optimization :-

→ old answer
→ poor values error
→ and based on
that making it
faster step size

→ Proposed in 1964

→ Idea: A ball is rolling down a hill - it's starts with very slow but picks up momentum and reaches terminal velocity.

- Momentum = $m \cdot v$ (mass • velocity)
- terminal velocity = Maximum attainable velocity during a free fall in a fluid or air.

Free fall



→ Air drag - it becomes a reverse force. So that

At some point of time,

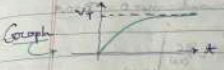
Drag force (F_D) and force (weight) which is pulling down to earth (mg)

These both force will becomes equal. That is called terminal velocity. (v_t)

Force direction



Another ball there will be some friction which will stop to slow down ball (like air) and when it reaches at bottom it will get terminal velocity.



→ It reaches terminal velocity by accumulating past change in velocity.

Similarly In our case.

Now gradient will be used to accelerate gradient descent.

For that,

We will introduce a new term which is called as Momentum.

i.e.

$$m \leftarrow p \cdot m + \eta \nabla_{\theta} J(\theta)$$

$$\theta \leftarrow \theta - m$$

In general $p=0.9$

assumption $\theta_0 = 0$

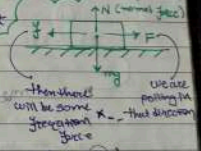
$$m = p \cdot m + \eta \frac{\partial J}{\partial w} \Big|_{w=w_{current}}$$

$$w = w - m$$

→ Simplified equation.

Analogy to coefficient of friction.

coefficient of momentum.



then there will be some friction force

and $f = \mu N$

coefficient of friction

If $\mu=0$,

then surface is smooth.

→ easy we can pull the box.

If $p=0$ then it is frictionless.

i.e. there is no accumulation.

Now,

$$m = p \cdot m + \eta \frac{\partial J}{\partial w} \Big|_{w=w_{current}}$$

$$\therefore m = \eta \frac{\partial J}{\partial w} \Big|_{w=w_{current}}$$

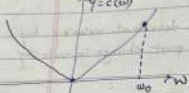
$$w = w - m$$

$$\therefore w = w - \eta \frac{\partial J}{\partial w} \Big|_{w=w_{current}}$$

→ Again it is nothing but gradient descent only.

Gradient descent is nothing but special case of momentum optimization.

Name: Observation: $y = c(w)$



Initially $w = w_0$ and $m = 0$, $\beta = 0.9$

Step 1

$$m = \beta m_0 + \eta \left. \frac{\partial c}{\partial w} \right|_{w=w_0}$$

$$\therefore m_1 = \eta \left. \frac{\partial c}{\partial w} \right|_{w=w_0} \quad (1)$$

Also, $w_1 = w_0 - m_1$

$$\therefore w_1 = w_0 - \eta \left. \frac{\partial c}{\partial w} \right|_{w=w_0}$$

(first weight updates)

[Gradient Descent only]

Step 2: $m_1 = \eta \left. \frac{\partial c}{\partial w} \right|_{w=w_0}$, $\beta = 0.9$

$$\text{Now, } m_2 = \beta m_1 + \eta \left. \frac{\partial c}{\partial w} \right|_{w=w_1}$$

$$= \beta \left(\eta \left. \frac{\partial c}{\partial w} \right|_{w=w_0} \right) + \eta \left. \frac{\partial c}{\partial w} \right|_{w=w_1}$$

$$\therefore m_2 = \eta \left[\beta \left. \frac{\partial c}{\partial w} \right|_{w=w_0} + \left. \frac{\partial c}{\partial w} \right|_{w=w_1} \right] \quad (3)$$

10.9 $\rightarrow$ post gradient $\rightarrow$ Present Gradient

$\rightarrow$ can say 90% of post gradient value taking

$$\rightarrow w_2 = w_1 - m_2$$

$$w_2 = w_1 - \left(\eta \left[\beta \left. \frac{\partial c}{\partial w} \right|_{w=w_0} + \left. \frac{\partial c}{\partial w} \right|_{w=w_1} \right] \right) \quad (A)$$

Special case,

If $\beta = 0$ then

$$w_2 = w_1 - \eta \left. \frac{\partial c}{\partial w} \right|_{w=w_1}$$

(B)

looks like gradient descent only.

Note

Also we can observe that in case (A), there will be higher weight updates.
 It is accumulating for the past weight updates as well. So we can say that step size will be big comparatively.

Step 3: $w = w_0$, $\beta = 0.9$, From eq (3) we have m_3 .

$$m_3 = \beta m_2 + \eta \frac{\partial C}{\partial w} \Big|_{w=w_2}$$

$$= \beta \left(\eta \left[\beta \frac{\partial C}{\partial w} \Big|_{w=w_0} + \frac{\partial C}{\partial w} \Big|_{w=w_1} \right] + \eta \frac{\partial C}{\partial w} \Big|_{w=w_2} \right)$$

In this case m_3 will be even larger.

$$m_3 = \eta \left(\underbrace{\beta^2 \frac{\partial C}{\partial w} \Big|_{w=w_0}}_{0.9 \times 0.9 \text{ 81\% of past gradients}} + \underbrace{\beta \frac{\partial C}{\partial w} \Big|_{w=w_1}}_{90\% of recent past gradient} + \underbrace{\frac{\partial C}{\partial w} \Big|_{w=w_2}}_{100\% of current gradient value} \right)$$

Now, $w_3 = w_2 - m_3$

and so on...

In this case we can see that if we go more step further, there we can see accumulation, less accumulation of past and more accumulation of recent past and so on.

Draw Backs:

→ One extra parameters β to tune, although $\beta = 0.9$ works fine in most of the cases but still it is an extra term.

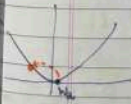
→ It oscillates when it reaches closer to local minima or global min. because of the accumulation of past momentum.

i.e. in $w_3 = \eta \left(\beta^2 \frac{\partial C}{\partial w} \Big|_{w=w_0} + \beta \frac{\partial C}{\partial w} \Big|_{w=w_1} + \frac{\partial C}{\partial w} \Big|_{w=w_2} \right)$

almost zero only

Suppose in graph w_2 is near to 0 only, so its derivative will be zero. Since, rest of two are from the past which all are not zero.
 So, $w_3 = w_2 - \eta (\text{non zero term})$

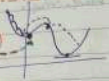
Means there will be weight update. → Most of the time it will overshoot.



So, this accumulation will impact, so gradually it will stop oscillating there near to zero.

Advantages:
 → Momentum helps in fast convergence.
 → Oscillatory can also help you to come out of local minima.

In only gradient descent it will stop over there



But in this case it will start oscillating, may be at certain point of time it will take big steps and jump from there.

There is chance that you may come out of your local minima. So that you can proceed further for global minima.

Keras: $\text{optimizer} = \text{tf.keras.optimizers.SGD}(\text{learning_rate} = 0.01, \text{momentum} = 0.9)$
 How to write this

* Nesterov Accelerated Gradient: (NAG) (1993)

- It is also known as Nesterov Momentum Optimization.
- It is faster than momentum optimization.

What is New?

It calculates gradient slightly ahead in the direction of momentum.

i.e. instead of calculating δ , it is calculating $(\delta - \beta m)$

Algorithm: $m \leftarrow \beta m + \eta \nabla \phi(\delta - \beta m)$

$\delta \leftarrow \delta - m$

avg

$$m = \beta m + \eta \frac{\partial C}{\partial w} \Big|_{w=(w-\beta m)}$$

$$W = W - m$$

assuming bias = 0,
otherwise with bias it
will be in vector form.

→ Initially $m=0$, $w=w_0$ and $\beta=0.9$

Step 1: $m_1 = \beta m_0 + \eta \frac{\partial C}{\partial w} \Big|_{w=(w_0-\beta m_0)}$

$$\therefore m_1 = \eta \frac{\partial C}{\partial w} \Big|_{w=w_0} \quad (1)$$

$$\rightarrow w_1 = w_0 - m_1$$

$$w_1 = w_0 - \eta \frac{\partial C}{\partial w} \Big|_{w=w_0}$$

Gradient
Descent

Put origin and etc
is there
i.e. (Wolt Value)
(It is -ve itself)

If i add some
+ve value into
-ve value it will
add more towards zero

Step 2: $w = w_1$, $m_0 = \text{equation (1)}$
and $\beta = 0.9$

$$m_2 = \beta m_1 + \eta \frac{\partial C}{\partial w} \Big|_{w=w_1}$$

→ So for now next step, instead of starting from w_1 point it will start slightly ahead of it.

→ is nothing but $(w_1 - \beta m_1)$ - this much value ahead.

$$\text{In that case, } m_2 = \beta \left(\eta \frac{\partial C}{\partial w} \Big|_{w=w_0} \right) + \eta \frac{\partial C}{\partial w} \Big|_{w=w_1 - \beta m_1}$$

$$= \eta \left(\beta \frac{\partial C}{\partial w} \Big|_{w=w_0} + \frac{\partial C}{\partial w} \Big|_{w=w_1 - \beta m_1} \right) \quad (2)$$

90% of past
gradient

100% gradient calculated

Slightly away from

Direction of move

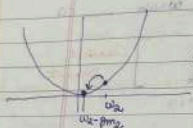
$$w_2 = w_1 - m_2$$

Step 3: while m_2 = equation 2, $\beta = 0.9$

$$m_2 = p m_1 + \eta \left. \frac{\partial C}{\partial w} \right|_{w=w_2 - p m_1}$$

$$m_2 = \eta \left[\beta \left. \frac{\partial C}{\partial w} \right|_{w=w_0} + \beta \left. \frac{\partial C}{\partial w} \right|_{w=w_1 - p m_1} + \left. \frac{\partial C}{\partial w} \right|_{w=w_2 - p m_2} \right]$$

90% of Past gradient 90% of Past recent gradient 100% current gradient



So, instead of calculating it from point w_2 it will start calculating from $(w_2 - p m_2)$

So, if we will calculate from $(w_2 - p m_2)$ then we will get gradient = 0.

Means, Base of this kind of term, last term will become zero.

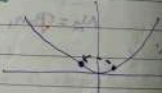
Means, in that case weight update won't happen.

So, in that case oscillation won't occur.

(jump)

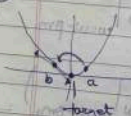
In other case, if it moves in other point then there slope will be negative.

i.e. $\left. \frac{\partial C}{\partial w} \right|_{w=w_2 - p m_1} = -ve$ means last term will be negative instead of zero.



Suppose, for n th term

$$m_n = \eta \left[\left(\text{Previous accumulator} \right) + \left. \frac{\partial C}{\partial m} \right|_{w=w(n-1) - p m(n-1)} \right]$$



at point a we are,

after some calculation it is overshooting and jumps into other side, that location slope is -ve.

Means,

In previous accumulator it will subtract some value that means weight updates will be lesser.

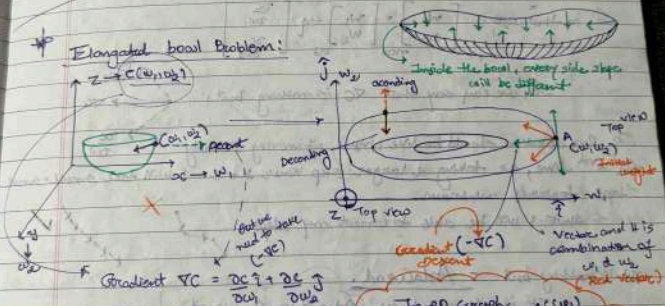
if it jumps because of overshooting

Advantages → It is faster than momentum optimization technique
 → oscillation is significantly reduced in this case.
 21.4 → Best of that it reaches close to local / global minima.

Disadvantages → Extra parameters to tune, i.e. β
 Best $\beta = 0.9$ works fine.

Keras: Optimizer = tf.keras.optimizers.SGD (learning rate = 0.01, momentum = 0.9, nesters = true)

Elongated bowl Problem:

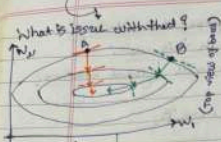


In something in this bowl,

(2D)
 Whatever point we start, it will always pointing away from the actual minima. so we need to do is Subtraction that is why it is decent and ascent.

In 2D concept
 Gradient w.r.t pointing always pointing away from the point
 It was always ascent and we were taking it was negative. $(-\nabla)$

If we increase the dimension we may face this (zick-zack) kind of problem



In case of point A, directly we can reach minimum point.
But in case of point B we can see that it is taking zick-zack path to reach at minimum. Directly it is not trying to reach at bottom.



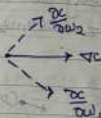
But in circular path / Diagrams that problem won't occur, we know that we won't get always a perfect cost function we may get this kind of structures



So, in that case,

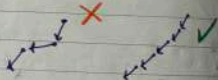
$$\begin{bmatrix} w_1 \\ w_2 \end{bmatrix}_{\text{new}} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}_{\text{old}} - \eta \begin{bmatrix} \frac{\partial C}{\partial w_1} \\ \frac{\partial C}{\partial w_2} \end{bmatrix}$$

So, we can say that ∇C is nothing but,



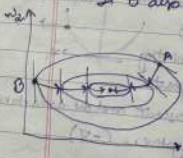
So our direction should be like, instead of moving directly in ∇C line, we taking a larger step over there, I will take small-small steps towards minimum.

So that I will be able to correct my steps.



It is nothing but - Adagrad.

- It is also called as Adaptive Gradient.



Previously we know, from the momentum algorithm

$$\begin{aligned} m &\leftarrow \beta m + \eta \nabla_\theta J(\theta) \\ \theta &\leftarrow \theta - m \end{aligned} \quad \left. \begin{array}{l} \text{Momentum} \\ \text{Algorithm} \end{array} \right\}$$

In case of NAG:

$$m \leftarrow \beta m + \eta \nabla_\theta J(\theta - \beta m)$$

classmate
Date _____
Page _____

→ Gradient is Vector, not Scalar value.

If both the (loss) we find that that change in weight (m) is proportional to gradient vector.

i.e. $\frac{\partial \mathcal{L}}{\partial w_1} + \frac{\partial \mathcal{L}}{\partial w_2}$, it is directly depends on these two values.

→ It doesn't care about directions issue.

So, Adaptive Gradient,

→ correct the direction in the initial steps only.

$$\begin{aligned} s &\leftarrow s + (\nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)) \\ \theta &\leftarrow \theta - \eta \nabla_{\theta} J(\theta) \oslash \sqrt{s + E} \end{aligned}$$

product of these two vectors
To avoid zero division error
Scaling factor
element wise dividing

Scaling factor $s = s + \left(\frac{\partial \mathcal{L}}{\partial w} \right)^2$

Weight update $w = w - \eta \frac{\frac{\partial \mathcal{L}}{\partial w}}{\sqrt{s + E}}$

η = Scaling factor
 E = Smoothing factor
i.e. $E = 10^{-10}$
 $\otimes$ = element wise multiply
 $\oslash$ = element wise division

Observation:

$$s \leftarrow s + [\nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)]$$

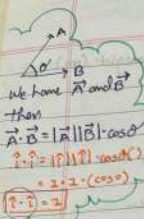
Both are vectors only

→ Gradient of: $\nabla f(x, y) = \frac{\partial f}{\partial x} \hat{i} + \frac{\partial f}{\partial y} \hat{j}$ (is a vector)

$$\begin{aligned} \text{Now, } \nabla f \cdot \nabla f &= \left(\frac{\partial f}{\partial x} \hat{i} + \frac{\partial f}{\partial y} \hat{j} \right) \cdot \left(\frac{\partial f}{\partial x} \hat{i} + \frac{\partial f}{\partial y} \hat{j} \right) \quad (\text{dot product}) \\ &= \left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2 \end{aligned}$$

$$|\nabla f| = \sqrt{\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2} \quad \text{--- (A)}$$

$\hat{i} \cdot \hat{i} = 1$
 $\hat{j} \cdot \hat{j} = 1$
 $\hat{i} \cdot \hat{j} = 0$



Now if θ is infinite
 or $\theta = 0$

Now, $\theta \leftarrow \theta - \eta \frac{(\nabla_{\theta} J(\theta))}{\sqrt{S + \epsilon}}$ (nearly zero)

i.e. $\theta \leftarrow \theta - \eta \frac{\nabla_{\theta} J(\theta)}{\sqrt{S}} \quad (B)$

Initially $S_0 = 0$

$\Rightarrow S_1 = S_0 + \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)$

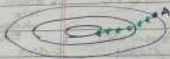
$\Rightarrow S_1 = S_0 + |\nabla_{\theta} J(\theta)|^2 \quad (\text{From eq (A)})$

Now, From eq (B),

$\theta \leftarrow \theta - \eta \frac{\nabla_{\theta} J(\theta)}{\sqrt{|\nabla_{\theta} J(\theta)|^2}}$

$\Rightarrow \theta \leftarrow \theta - \eta \frac{\nabla_{\theta} J(\theta)}{|\nabla_{\theta} J(\theta)|} \quad \left(\text{i.e. } \frac{\text{vector}}{|\text{vector}|} = \text{Unit Vector} \right)$

Unit Vector



here starting with point A, there we won't take large step, we will take very small - Unit Step that will multiplied (learning rate)

i.e. $w = w - \eta |\text{Unit vector}| \quad (\text{Baby step})$

More Discretization:

$S = S + \left(\frac{\partial C}{\partial w} \right)^2$ and $w = w - \eta \frac{\left(\frac{\partial C}{\partial w} \right)}{\sqrt{S + \epsilon}}$ (Magnitude of vector)

Step 1: $S = 0, w = w_0, \eta = 0$

$\Rightarrow S_1 = S_0 + \left(\frac{\partial C}{\partial w} \bigg|_{w=w_0} \right)^2 \Rightarrow S_1 = \left(\frac{\partial C}{\partial w} \bigg|_{w=w_0} \right)^2$

$\Rightarrow w_1 = w_0 - \eta \frac{\frac{\partial C}{\partial w} \big|_{w=w_0}}{\sqrt{S_1 + \epsilon}}$

$\Rightarrow w_1 = w_0 - \eta \frac{\frac{\partial C}{\partial w} \big|_{w=w_0}}{\sqrt{\left(\frac{\partial C}{\partial w} \big|_{w=w_0} \right)^2 + \epsilon}} \quad (\text{first weight update})$

$$\approx \frac{\eta \left(\frac{\partial C}{\partial w} \right)_{w=w_0}}{\sqrt{\left(\frac{\partial C}{\partial w} \right)_{w=w_0}^2 + 1}}$$

$$\approx \frac{w_0 - \eta \nabla C}{\sqrt{1 + \eta^2}}$$

Step 2:

$$S_1 = \left(\frac{\partial C}{\partial w} \right)_{w=w_0}, w=w_0$$

$$\rightarrow S_2 = S_1 + \left(\frac{\partial C}{\partial w} \right)_{w=w_1}$$

new updates

$$\xrightarrow{\text{first updates}} = \left(\frac{\partial C}{\partial w} \right)_{w=w_0} + \left(\frac{\partial C}{\partial w} \right)_{w=w_1}$$

$$\rightarrow w_2 = w_1 - \eta \frac{\partial C}{\partial w} \bigg|_{w=w_1}$$

(here denominator $\geq |\text{vector}|^2$)

(overall magnitude ≤ 1)

denominator will become slightly larger

$$\sqrt{S_2 + \epsilon}$$

Also, gradient is adapting as per

the denominators.

here,

$$\Delta w \propto \frac{\frac{\partial C}{\partial w} \big|_{w=w_j}}{\sqrt{\sum_{i=0}^{N-1} \left(\frac{\partial C}{\partial w} \big|_{w=w_i} \right)^2 + \epsilon}}$$

about n^{th} term.

→ Called as Levenberg

Also, we can say, for same weight updates,

$$w = w - \eta \frac{\frac{\partial C}{\partial w} \big|_{w=w_0}}{K}$$

$$\text{i.e. } w = w - \frac{\eta}{K} \frac{\partial C}{\partial w} \bigg|_{w=w_0} \quad \left(\text{where, } K = \sqrt{\sum_{i=0}^{N-1} \left(\frac{\partial C}{\partial w} \big|_{w=w_i} \right)^2 + \epsilon} \right)$$

And now, if i compare it with gradient descent,

$$w = w - \eta \frac{\partial C}{\partial w} \bigg|_{w=w_0}$$

, can see that, that 1st term is

Here,

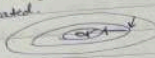
→ learning rate is decaying, denominator keeps on increasing means

overall value is decreasing

That is called as Decay in learning rate by factor of K.

GD

→ It moves quickly in the direction of steepest descent without the considering where global minima is located.

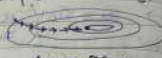


$\Delta W \propto \frac{\partial C}{\partial W}$

→ $W = W - \eta \left(\frac{\partial C}{\partial W} \right)$

AdaGrad

→ It corrects with its direction by scaling down the gradient vector along the steepest direction.



$\Delta W \propto \frac{\partial C}{\partial W} / \sqrt{K}$, $K = 1, 2, \dots$

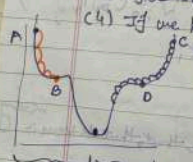
→ $W = W - \frac{\eta}{K} \left(\frac{\partial C}{\partial W} \right)$

Advantages:

- (i) It corrects the direction initially unlike gradient descent.
 - (ii) Less tuning of learning parameters.
- Before we were experimenting that model for that in weight updation, we were trying diff learning rates.

- Disadvantages:
- (1) Stops early before reaching global minima. (∵ Decay η)
 - (2) Takes longer time to converge due to decaying learning rate.
 - (3) That's why NOT RECOMMENDED TO USE

- (3) After taking longer time also it is not converging.
- (4) If we have curve like this,



At point A, GB will take larger steps, and at saddle point B it will slow down over there.

→ It's saddle point, neither local or global minima.

at that point $\frac{\partial C}{\partial W} = 0$, i.e. $\Delta W \approx 0$

ie. Almost very less weight updates.